Audit Report

January 8, 2026

Shielded Technologies - Midnight Reserve

Contents

1 - Summary ... 3
1.a - Overview ... 3
1.b - Scope ... 4
1.c - Process ... 4
2 - Audited Files ... 5
3 - Specification ... 6
3.a - UTxOs ... 6
3.b - Transactions ... 7
4 - Findings ... 34
4.a - MRC-001 Merge logic allows the creation of unspendable outputs ... 35
4.b - MRC-002 “cNIGHT Generates Dust” NFTs can be extracted from contract ... 36
4.c - MRC-003 Too restrictive logic round version check ... 37
4.d - MRC-201 Missing datum checks in the Main Gov Threshold UTxO ... 38
4.e - MRC-202 Merge logic allows the inclusion of arbitrary assets ... 39
4.f - MRC-203 Permissioned contracts can contain incorrect datum ... 40
4.g - MRC-204 Staging Auth Mitigation Script involves running previous undesired AMS ... 41
4.h - MRC-301 No inline documentation ... 42
4.i - MRC-302 Modularizable code for value checks ... 43
4.j - MRC-303 Multisig authorization approach mints useless tokens ... 44
5 - Minor issues ... 45
A Appendix ... 45
A.1 Terms and Conditions of the Commercial Agreement ... 45
A.2 Issue Guide ... 48
A.3 Revisions ... 49
A.4 About Us ... 49

1 - Summary

This report provides a comprehensive audit of the Midnight Reserve project, which contains the on-chain code for the Cardano-side implementation of the Midnight Protocol. It includes, but is not limited to, the NIGHT token reserve and a library for upgrading contract scripts.

Not all components of the project were audited, but a significant subset was audited based on a prior agreement with the client. The complete list of audited files can be found in Section 3.

The investigation spanned several potential vulnerabilities, including scenarios where attackers might exploit the validator to lock up or steal funds.

The audit is conducted without warranties or guarantees of the quality or security of the code. It's important to note that this report only covers identified issues, and we do not claim to have detected all potential vulnerabilities.

1.a - Overview

The Midnight Reserve project contains the on-chain code for the Cardano-side implementation of the Midnight Protocol.

One of the main components is the **Reserve**, which serves as a container of NIGHT tokens controlled by a governance entity. The logic of this contract is upgradeable. Therefore, the Reserve is implemented on top of a complete library for upgradable contracts included in the project.

The **upgradable library** encapsulates the contract logic within withdrawal scripts. The currently active logic is stored in a *main* datum. Updates to this logic are authorized by an *auth* withdrawal script, which is also stored in the *main* datum and is itself upgradable. For additional safety, logic and auth scripts must first be staged in a secondary *staging* datum before being updated. This allows the scripts to be tested prior to being promoted to the *main* datum.

Default authorization logics are provided for both staging and promoting scripts. These logics are based on several governance mechanisms included in the project, namely the Council, the Technical Authority, and the Thresholds.

The **Council** and the **Technical Authority** (TA) lock, each into a single UTxO, a datum containing the composition of the authority as a list of signers. Both contract are themselves upgradable, meaning that the logic used to modify the authorities can also be upgraded. An initial logic script for updating the signers is provided, and is likewise based on the governance mechanisms mentioned above: the Council, the Technical Authority and the Thresholds.

The **Thresholds** contracts lock datums in different UTxOs that specify the proportion of Council and TA members required to authorize particular actions. Two specific instances are referenced by the default authorization scripts to define the thresholds for staging and promoting logic in the upgradable contracts. Two other instances define the thresholds for updating the composition of the Council and the TA.

The Midnight Reserve project also includes several other contracts that are less tightly coupled with the main architecture: Federated Ops, Terms and Conditions, Illiquid Circulation Supply, cNight Generates Dust and Committee Bridge.

Federated Ops is an upgradable contract that defines a third governance entity. This entity does not have a role in authorizing actions in any other contract in the project. The initial logic for updating the

composition of Federated Ops requires approval from both the Council and the TA, with thresholds defined by a specific instance of the Thresholds contract.

Terms and Conditions is an upgradable contract intended for informational purposes. As with the three governance entities, the initial logic for updating this information involves the Council and the TA, with thresholds defined by a specific instance of the Thresholds contract.

The **Illiquid Circulation Supply (ICS)** is also an upgradable contract and is similar to the Reserve in all aspects of its initial logic. In practice, the ICS is intended to serve a different purpose than the Reserve. However, this distinction is not inferrable from the current implementation.

cNight Generates Dust is a completely standalone contract that allows NIGHT holders on Cardano to register Midnight addresses for receiving DUST. UTxOs locked under this contract store, in their datum, a pairing of a Cardano staking credential with a Midnight address.

The **Committee Bridge** is a separate set of contracts that fall outside the scope of this audit.

1.b - Scope

This audit focuses primarily on security issues. It also covers other aspects such as code quality, optimization and documentation. Functionality is only partially reviewed on a best-effort basis. The auditing team can only provide limited feedback, based on their experience and understanding of the business uses cases.

Under no circumstances this audit can replace the necessary steps required in a software development process to validate the functional requirements underlying the current design and implementation. These steps include, but may not be limited to, beta-testing with potential end-users.

Some functional aspects we would like to highlight as requiring validation are:

- The need for a staging phase to enable testing on mainnet, rather than relying directly on preview/preprod.
- The one-shot approach for mitigation scripts, which can be set only once and permanently.
- All integrations with the Midnight protocol, including Midnight keys and addresses.

1.c - Process

Our audit process involved a thorough examination of Midnight Reserve validators. Areas vulnerable to potential security threats were closely scrutinized, including those where attackers could exploit the validator's functions to disrupt the platform and its users. This included evaluating potential risks such as unauthorized asset addition, hidden market creation, and disruptions to interoperability with other Plutus scripts. This also included the common vulnerabilities such as double satisfaction and minting policy vulnerabilities.

The audit took place over a period of several weeks, and it involved the evaluation of the protocol's mathematical model to verify that the implemented equations matched the expected behavior.

Findings and feedback from the audit were communicated regularly to the Shielded Technologies team through Slack. Diagrams illustrating the necessary transaction structure for proper interaction with the protocol are attached as part of this report. The Shielded Technologies team addressed these issues in an efficient and timely manner, enhancing the overall security of the platform.

2 - Audited Files

Below is a list of all audited files in this report. Any files **not** listed here were **not** audited. The final state of the files for the purposes of this report is considered to be commit `f11d27828666e887fb495a85242edf9b8a78192f`.

Filename
./validators/permissioned.ak
./validators/terms_and_conditions.ak
./validators/thresholds.ak
./validators/reserve.ak
./validators/cnight_generates_dust.ak
./validators/illiquid_circulation_supply.ak
./validators/gov_auth.ak
./lib/auth/types.ak
./lib/auth/staging.ak
./lib/auth/script.ak
./lib/upgradable/types.ak
./lib/upgradable/two-stage-upgrade.ak
./lib/utils.ak
./lib/logic/script.ak
./lib/forever/types.ak
./lib/forever/script.ak
./lib/multisig/types.ak
./lib/multisig/script.ak

3 - Specification

3.a - UTxOs

3.a.a - Reserve

3.a.a.a - Reserve UTxO

Address: Hash of `reserve_forever`

Implicit parameters:

- One shot UTxO reference:
  - `config.reserve_one_shot_hash`
  - `config.reserve_one_shot_index`
- Reserve Governance script hash (hash of `reserve_two_stage_upgrade`):
  - `config.reserve_two_stage_hash`
- CNIGHT policy:
  - `config.cnight_policy`

3.a.a.b - Main Reserve Governance UTxO

Address: Hash of `reserve_two_stage_upgrade`

Implicit parameters:

- One shot UTxO reference:
  - `config.reserve_one_shot_hash`
  - `config.reserve_one_shot_index`
- CNIGHT policy:
  - `config.cnight_policy`

3.a.a.c - Staging Reserve Governance UTxO

Address: Hash of `reserve_two_stage_upgrade`

Implicit parameters:

- One shot UTxO reference:
  - `config.reserve_one_shot_hash`
  - `config.reserve_one_shot_index`
- CNIGHT policy:
  - `config.cnight_policy`

3.a.b - Council

- Council UTxO
- Main Council Governance UTxO
- Staging Council Governance UTxO
- Council Update Threshold UTxO

3.a.c - Technical Authority

- Technical Authority UTxO
- Main Technical Authority Governance UTxO
- Staging Technical Authority Governance UTxO
- Technical Authority Update Threshold UTxO

3.a.d - Federated Authority

- Federated Authority UTxO
- Main Federated Authority Governance UTxO

- Staging Federated Authority Governance UTxO
- Federated Authority Update Threshold UTxO

3.a.e - Default/Initial Auth

- Main Auth Threshold UTxO
- Staging Auth Threshold UTxO

3.b - Transactions

The main objective of the transaction diagrams that follow is to outline the key aspects of each operation in the procotol. To provide a clearer presentation, we will omit details such as the minimum amount of ADA in each UTxO, fees paid, and the sources of that ADA, etc.

3.b.a - Reserve

Involved scripts:

- `reserve_forever` script
- `reserve_two_stage_upgrade` (`reserve_TSU`) script

Involved Addresses:

- reserve_two_stage_addr = { payment: `reserve_TSU`, staking: any }
- reserve_addr = { payment: `reserve_forever`, staking: any }

Involved Assets:

- main_reserve_TSU_NFT = `reserve_TSU.main`
- staging_reserve_TSU_NFT = `reserve_TSU.staging`
- reserve_NFT = `reserve_forever.""`

3.b.a.a - Initalize Reserve

Involved scripts:

`reserve_forever` script

- Mint
  - Redeemer: `()`

`reserve_two_stage_upgrade` script

- Mint
  - Redeemer: `()`

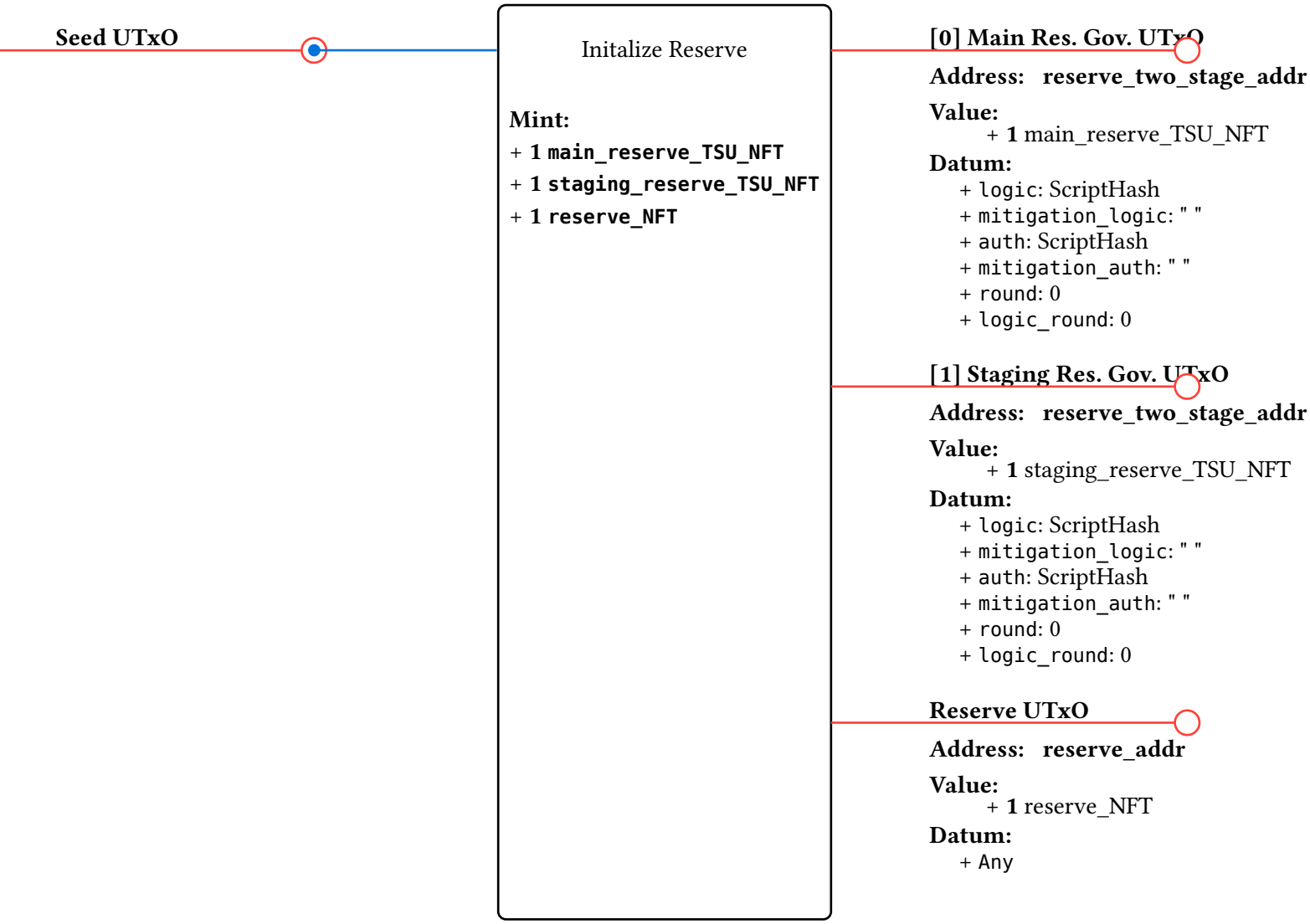


Figure 1: Initalize Reserve transaction

<<<<<<< Updated upstream

3.b.a.a.-.-.a -

3.b.a.b - Invoke Reserve

Involved scripts:

- Spend Main Reserve UTxO (`reserve_forever` script) (REDEEMER?)
- "Logic" withdrawal: read from Main Governance UTxO datum
- "Mitigation logic" withdrawal: if not empty, read from Main Governance UTxO datum

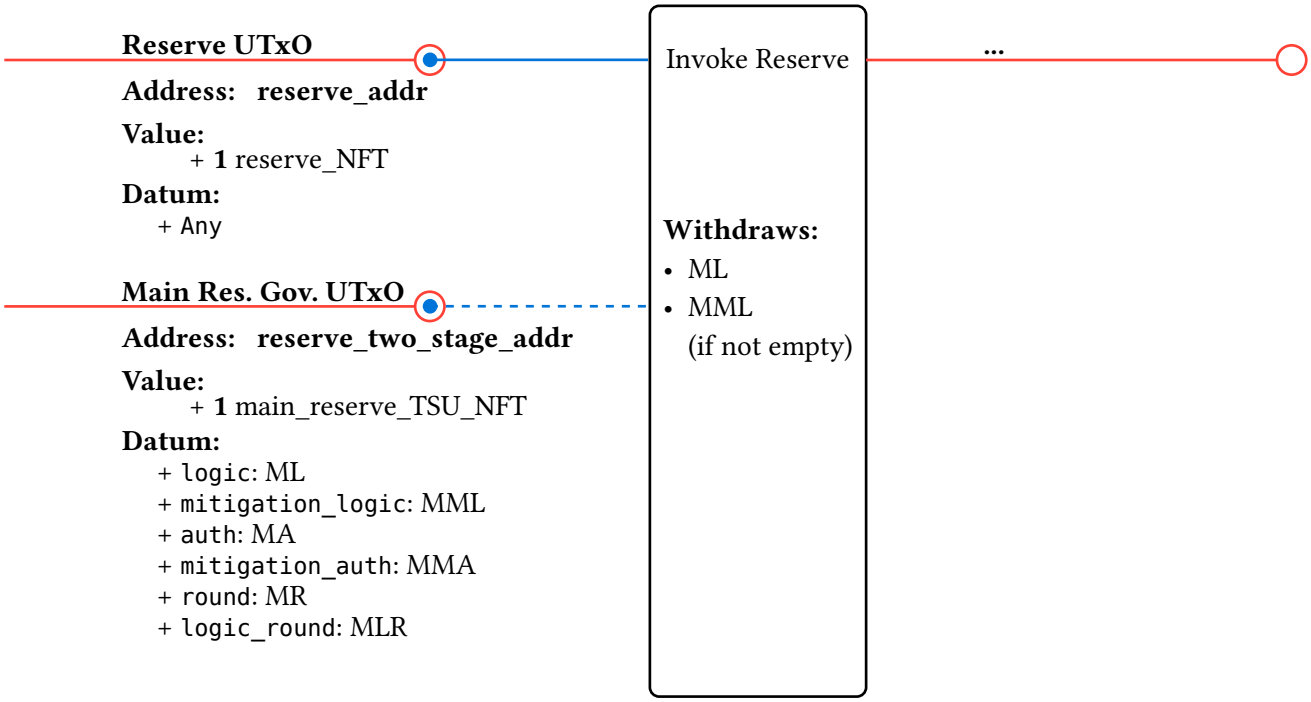


Figure 2: Invoke Reserve transaction

>>>>>>> Stashed changes

3.b.a.c - Invoke Reserve with Initial Reserve Logic

Special case of "Invoke Reserve" when `logic` in Main Governance UTxO datum is set to `reserve.ak:reserve_logic()` script.

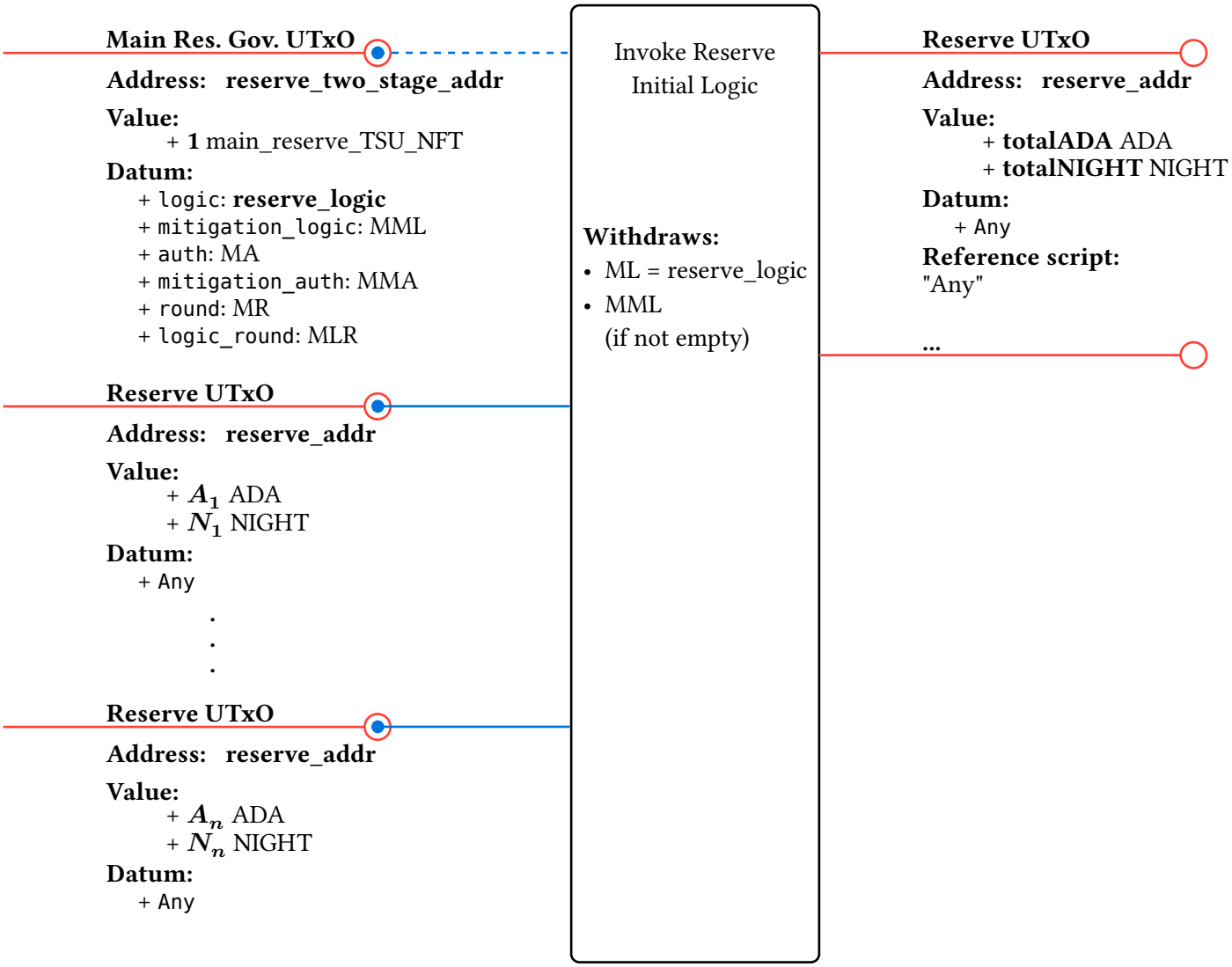
Involved scripts:

`reserve_logic` script

- Withdraw
  - Redeemer: `()`

`reserve_forever` script

- Spend
  - Redeemer: `()`



Note:

The input for each Reserve UTxO may contain other tokens, but the output must only include ADA and NIGHT tokens

$$\sum_{i=1}^{n} A_i \leq \text{totalADA}$$

$$\sum_{i=1}^{n} N_i \leq \text{totalNIGHT}$$

Figure 3: Invoke Reserve transaction with logic given by given `reserve.ak:reserve_logic()`.

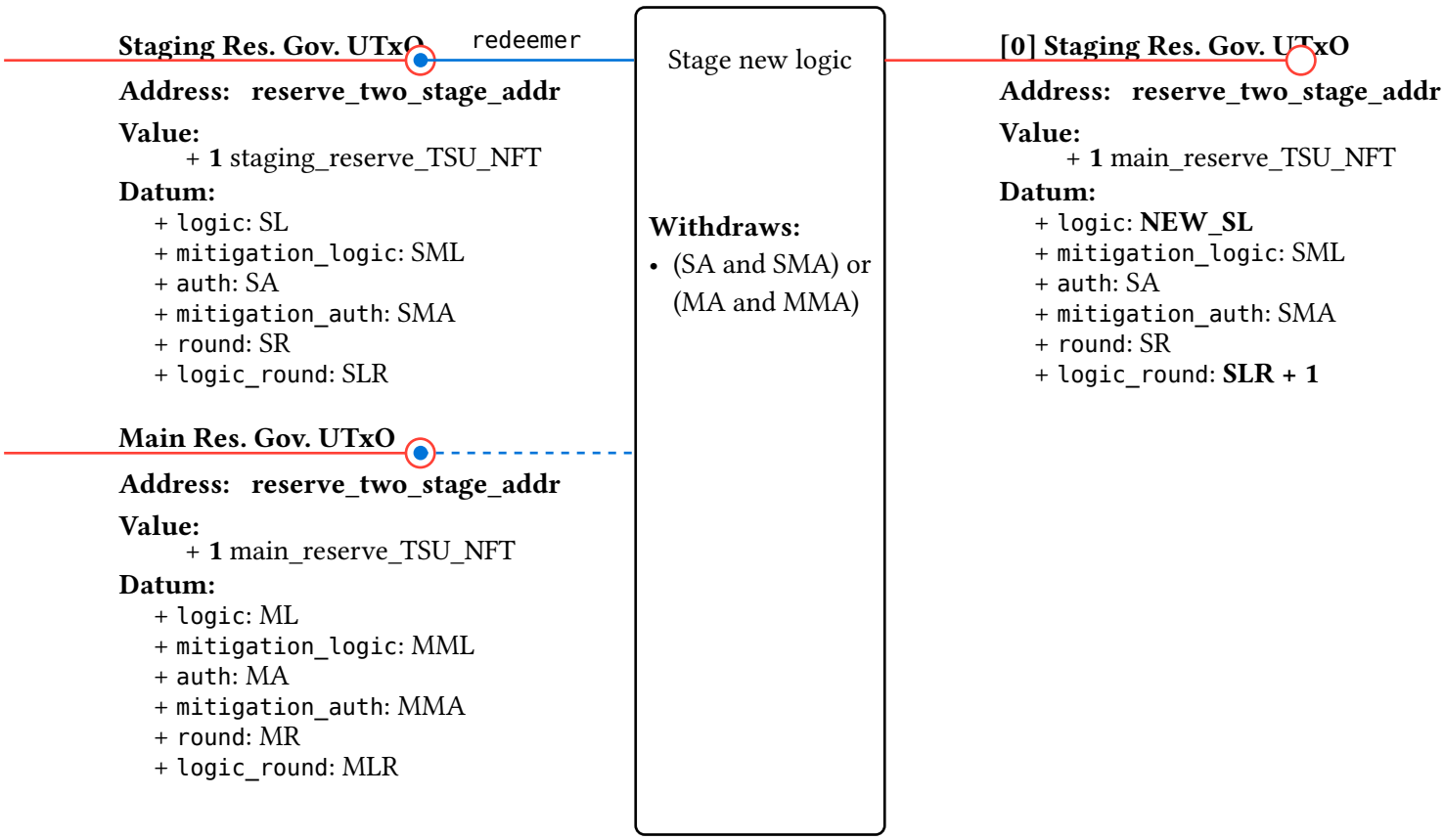
3.b.a.d - Stage New Logic

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ Logic, Staging("Main Res. Gov. UTxO", new_script_hash) }`
- Withdrawals: At least one of these two options:
  - Staging auth and mitigation auth (if not empty)
  - Main auth and mitigation auth (if not empty)

TODO: If auth logics are those provided in `validators/gov_auth.ak`, then there are additional requirements: Council and Tech Auth must approve, following thresholds defined by Main/Staging Auth Threshold UTxOs.



Note:

`redeemer` = `TwoStageRedeemer{Logic, Staging("Main Res. Gov. UTxO", NEW_SL)}`

Figure 4: Stage new logic transaction

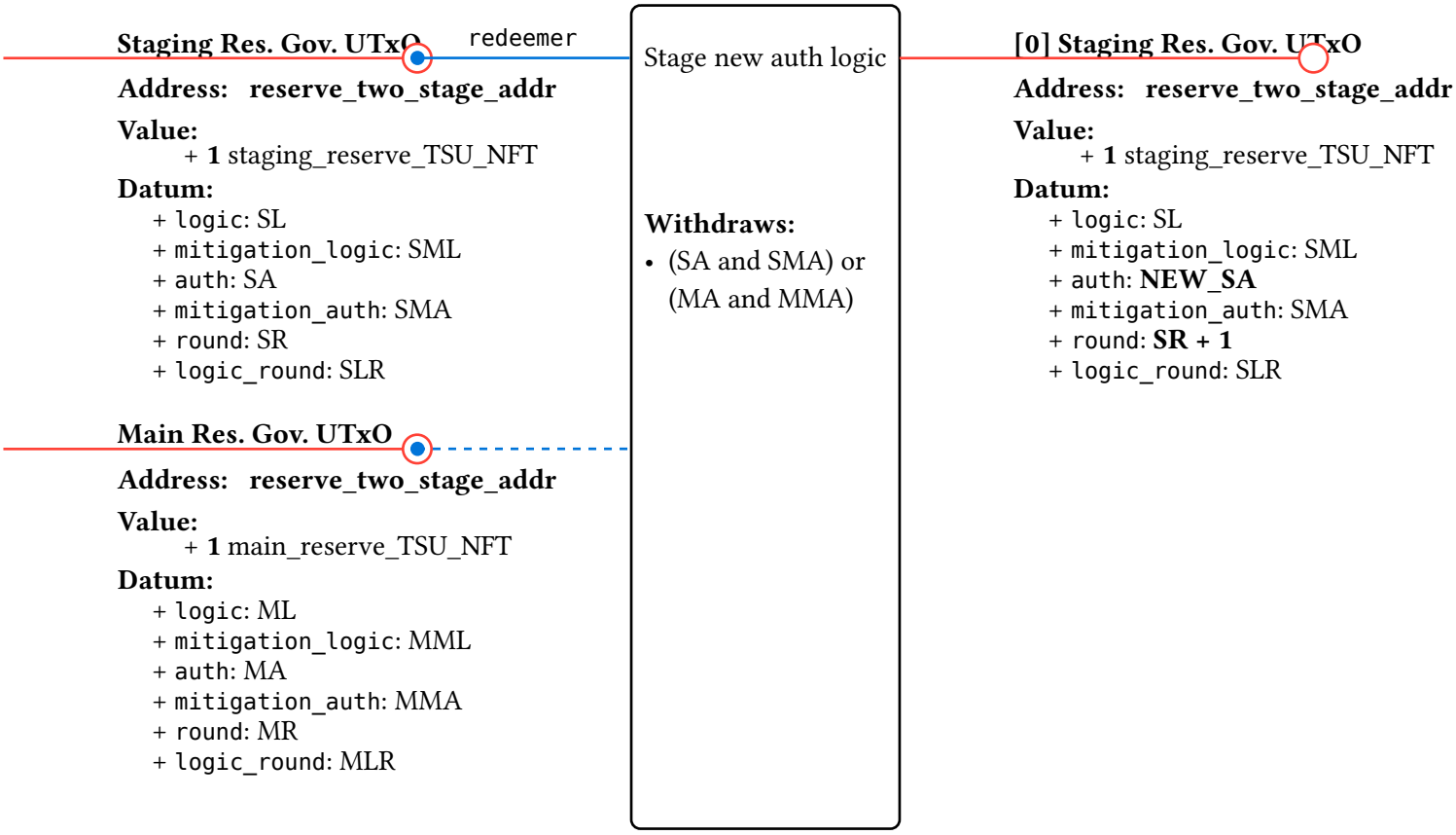
3.b.a.e - Stage New Auth

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ Auth, Staging("Main Res. Gov. UTxO", new_script_hash) }`
- Withdrawals: At least one of these two options:
  - Staging auth and mitigation auth (if not empty)

- Main auth and mitigation auth (if not empty)



Note:

`redeemer = TwoStageRedeemer{Auth, Staging("Main Res. Gov. UTxO", NEW_SA)}`

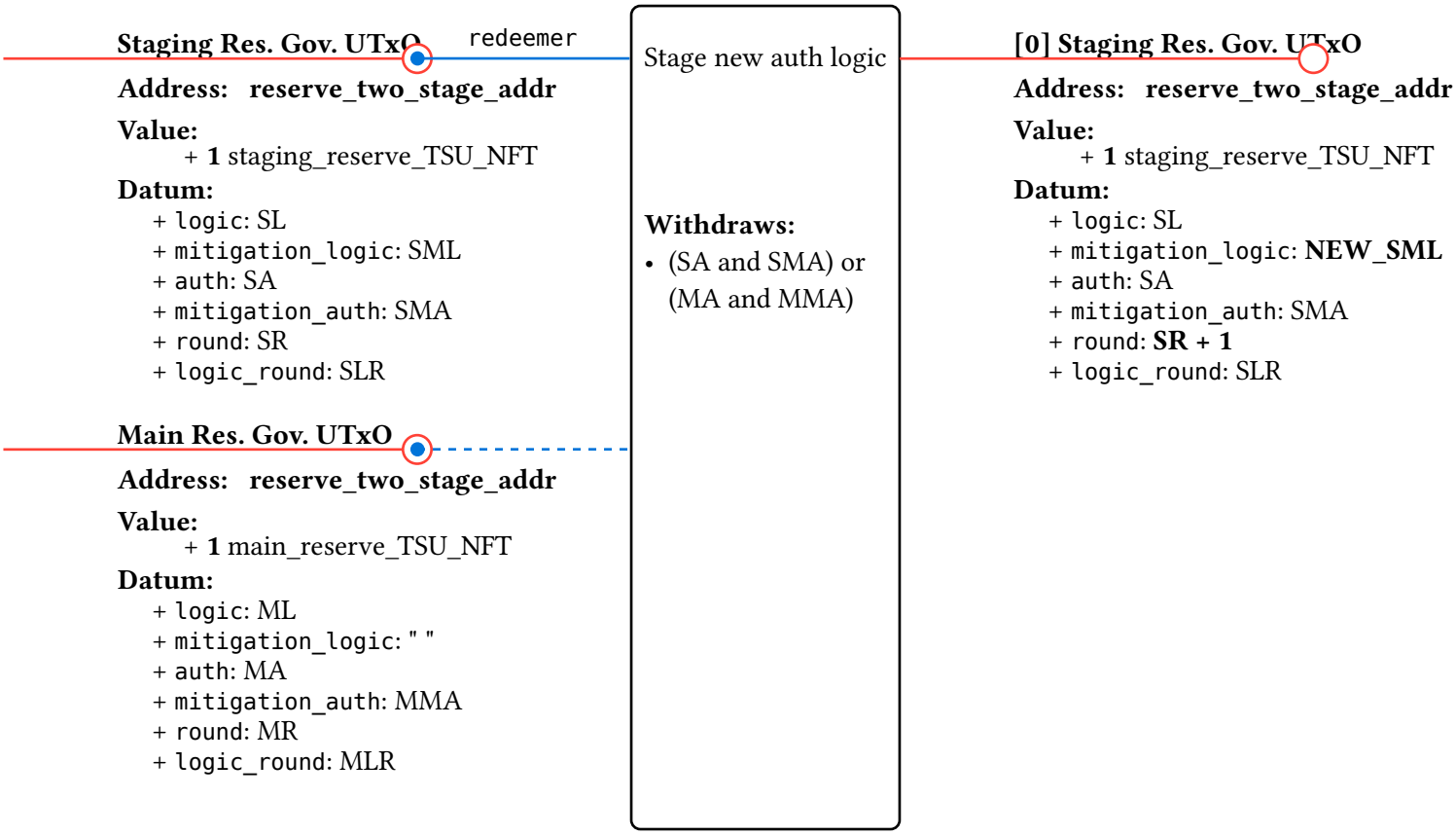
Figure 5: Stage new auth transaction

3.b.a.f - Stage Mitigation Logic

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ MitigationLogic, Staging("Main Res. Gov. UTxO", new_script_hash) }`
- Withdrawals: At least one of these two options:
  - Staging auth and mitigation auth (if not empty)
  - Main auth and mitigation auth (if not empty)



Note:

`redeemer` = `TwoStageRedeemer{MitigationLogic, Staging("Main Res. Gov. UTxO", NEW_SML)}`

The `mitigation_logic` from the "Main Res. Gov. UTxO" datum must be not seted

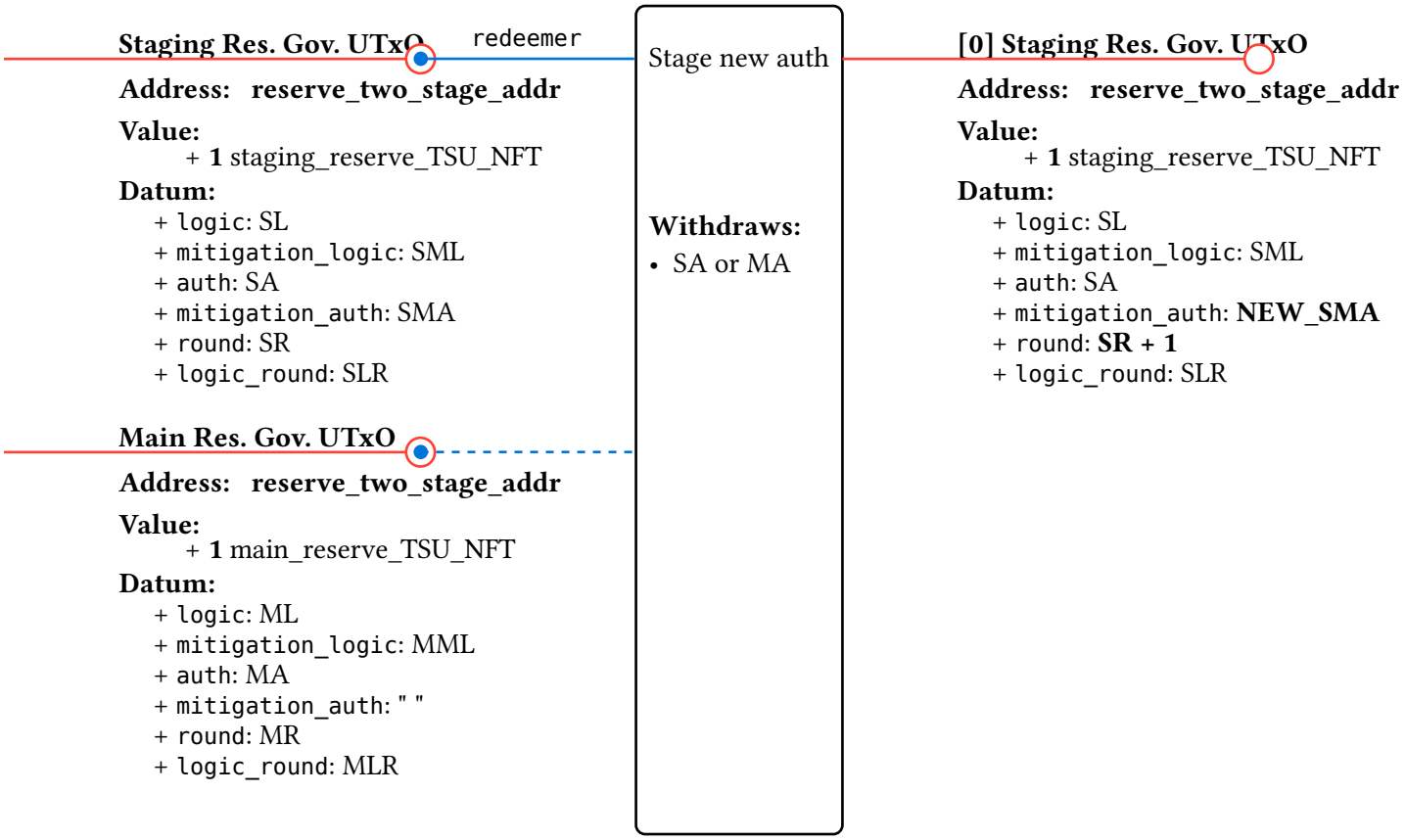
Figure 6: Stage new mitigation logic transaction

3.b.a.g - Stage Mitigation Auth

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ MitigationLogic, Staging("Main Res. Gov. UTxO", new_script_hash) }`
- Withdrawals: At least one of these two options:
  - Staging auth
  - Main auth



Note:

`redeemer` = `TwoStageRedeemer{MitigationAuth, Staging("Main Res. Gov. UTxO", NEW_SMA)}`

The `mitigation_auth` from the "Main Res. Gov. UTxO" datum must be not seted

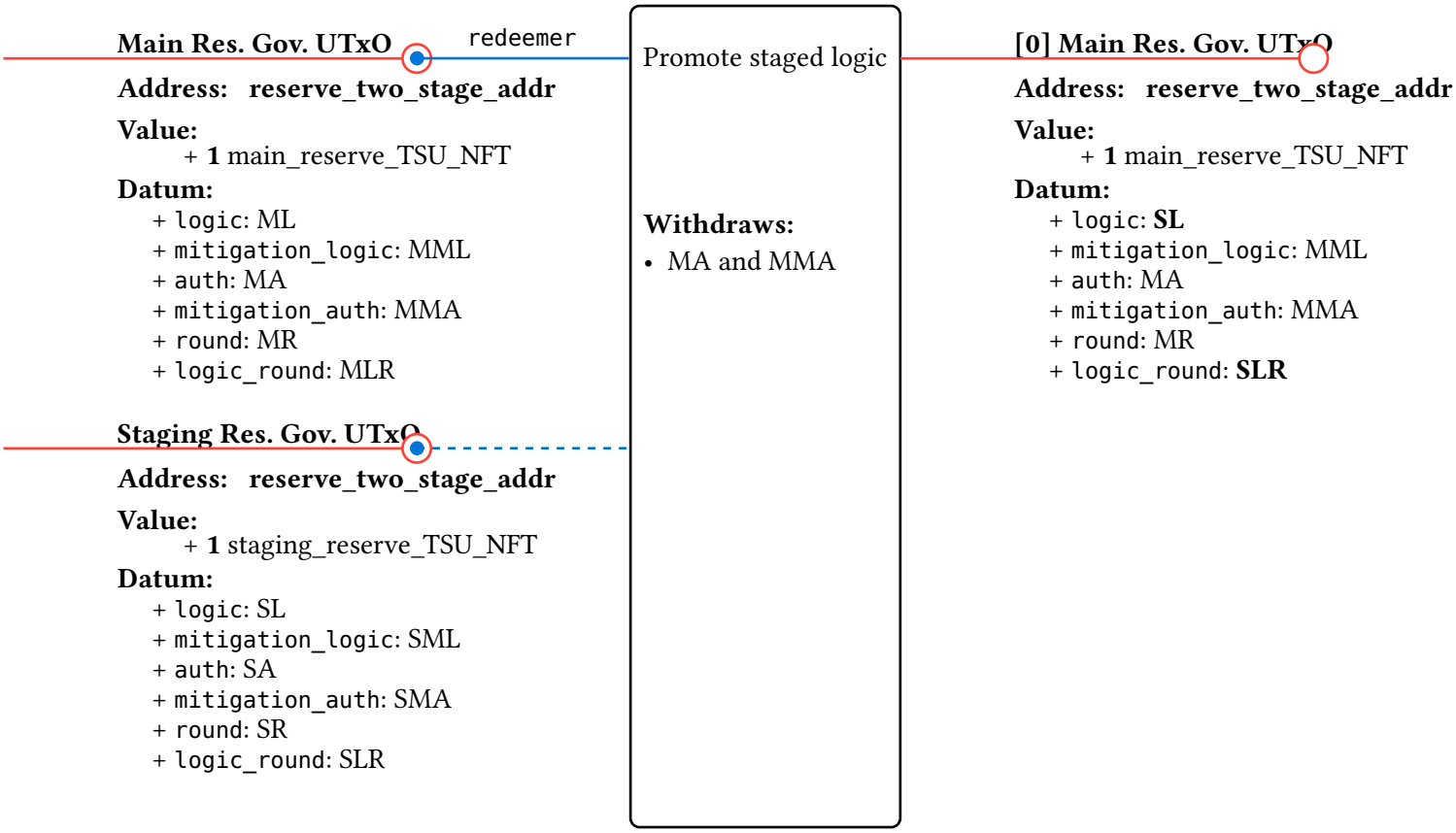
Figure 7: Stage new mitigation logic transaction

3.b.a.h - Promote Staged Logic

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ Logic, Main("Staging Res. Gov. UTxO") }`
- Withdrawals:
  - Main auth and mitigation auth (if not empty)



Note:

`redeemer = TwoStageRedeemer{Logic, Main("Staging Res. Gov. UTxO")}`

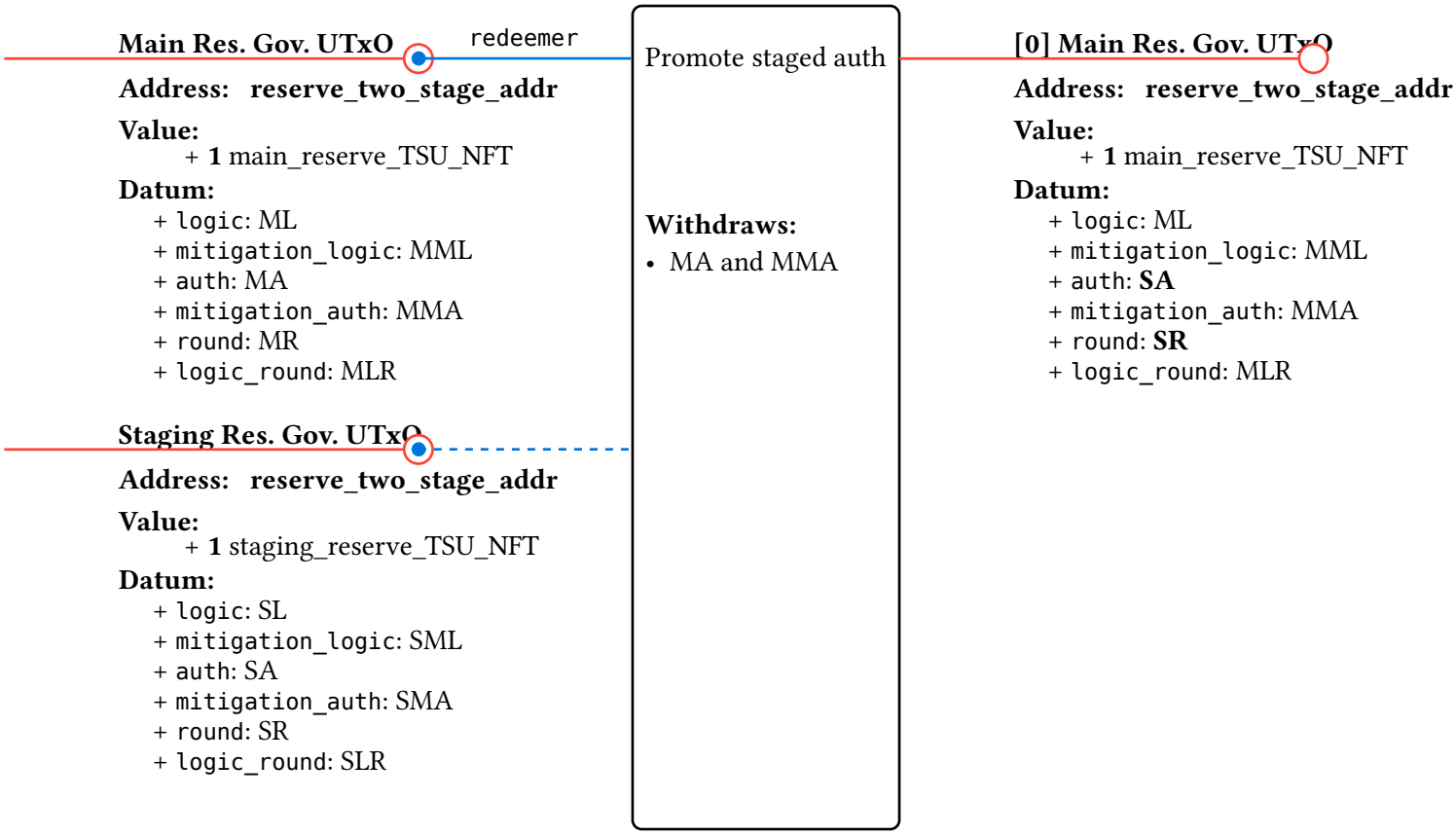
Figure 8: Promote staged logic

3.b.a.i - Promote Staged Auth

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ Auth, Main("Staging Res. Gov. UTxO") }`
- Withdrawals:
  - Main auth and mitigation auth (if not empty)



Note:

`redeemer` = `TwoStageRedeemer{Auth, Main("Staging Res. Gov. UTxO")}`

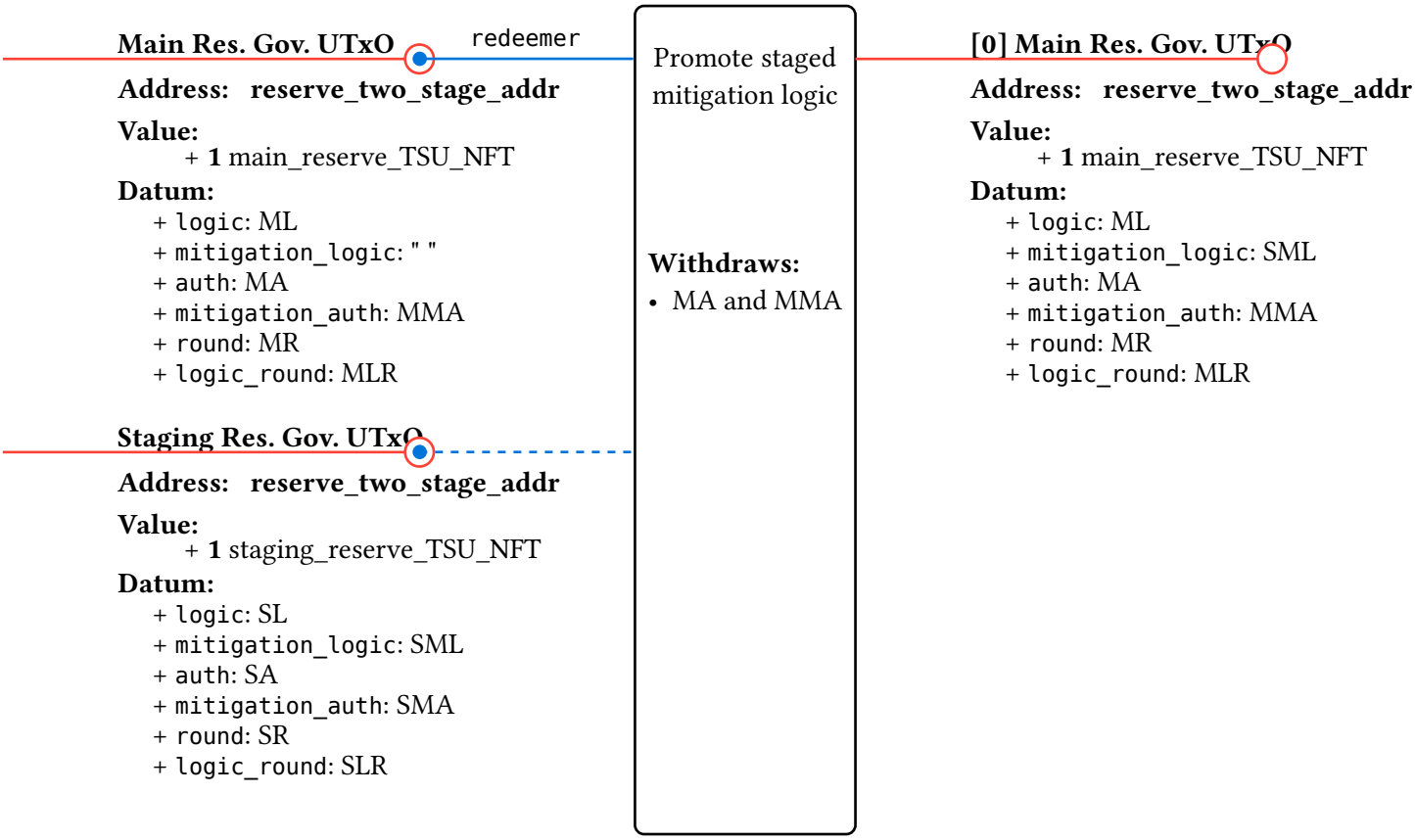
Figure 9: Promote staged auth

3.b.a.j - Promote Mitigation Logic

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ MitigationLogic, Main("Staging Res. Gov. UTxO") }`
- Withdrawals:
  - Main auth and mitigation auth (if not empty)



Note:

`redeemer = TwoStageRedeemer{MitigationLogic, Main("Staging Res. Gov. UTxO")}`

The `mitigation_logic` from the "Main Res. Gov. UTxO" datum must be not seted

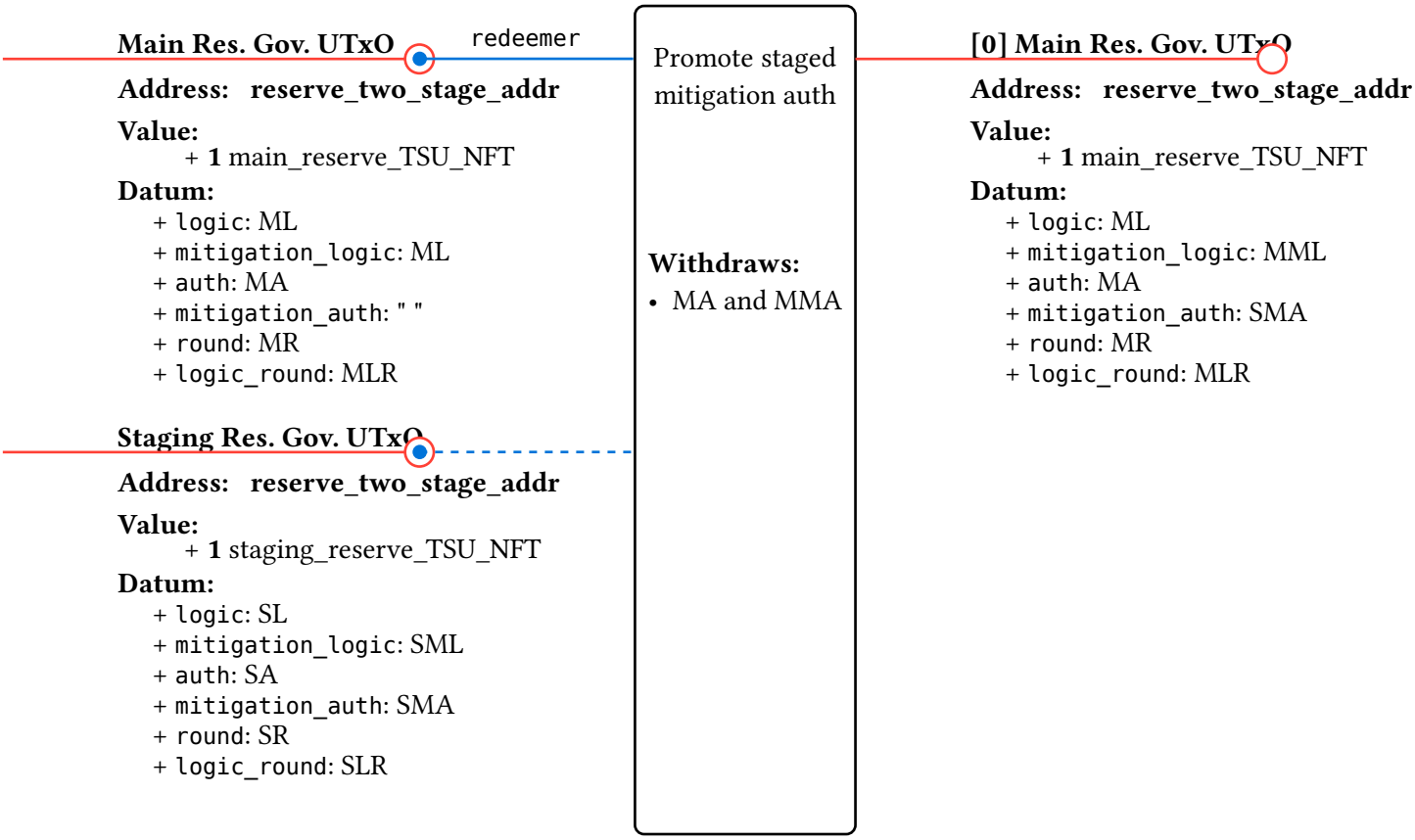
Figure 10: Promote staged mitigation logic

3.b.a.k - Promote Mitigation Auth

Involved scripts:

`reserve_two_stage_upgrade` script

- Spend
  - Redeemer: `TwoStageRedeemer{ MitigationAuth, Main("Staging Res. Gov. UTxO") }`
- Withdrawals:
  - Main auth and mitigation auth (if not empty)



Note:

`redeemer = TwoStageRedeemer{MitigationAuth, Main("Staging Res. Gov. UTxO")}`

The `mitigation_auth` from the "Main Res. Gov. UTxO" datum must be not seted

Figure 11: Promote staged mitigation auth

3.b.b - Council

Involved scripts:

- `council_forever` script
- `council_two_stage_upgrade` (`council_TSU`) script
- `tech_auth_forever` script
- `main_gov_threshold` (`mainGT`) script.

Involved Addresses:

- council_two_stage_addr = { payment: `council_TSU`, staking: any }
- council_addr = { payment: `council_forever`, staking: any }
- tech_auth_addr = { payment: `council_forever`, staking: any }
- mainGT_addr = { payment: `mainGT`, staking: any }

Involved Assets:

- main_council_TSU_NFT = `council_TSU.main`
- staging_council_TSU_NFT = `council_TSU.staging`
- council_NFT = `council_forever.""`
- tech_auth_NFT = `council_forever.""`
- mainGT_token = `mainGT.""`

3.b.b.a - Initalize Council

Involved scripts:

`reserve_forever` script

- Mint The provided redeemer is checked to be compatible with the output Council UTxO datum using function `permissioned.ak:validate_multisig_structure()`.

`reserve_two_stage_upgrade` script

- Mint
  - Redeemer: `()`

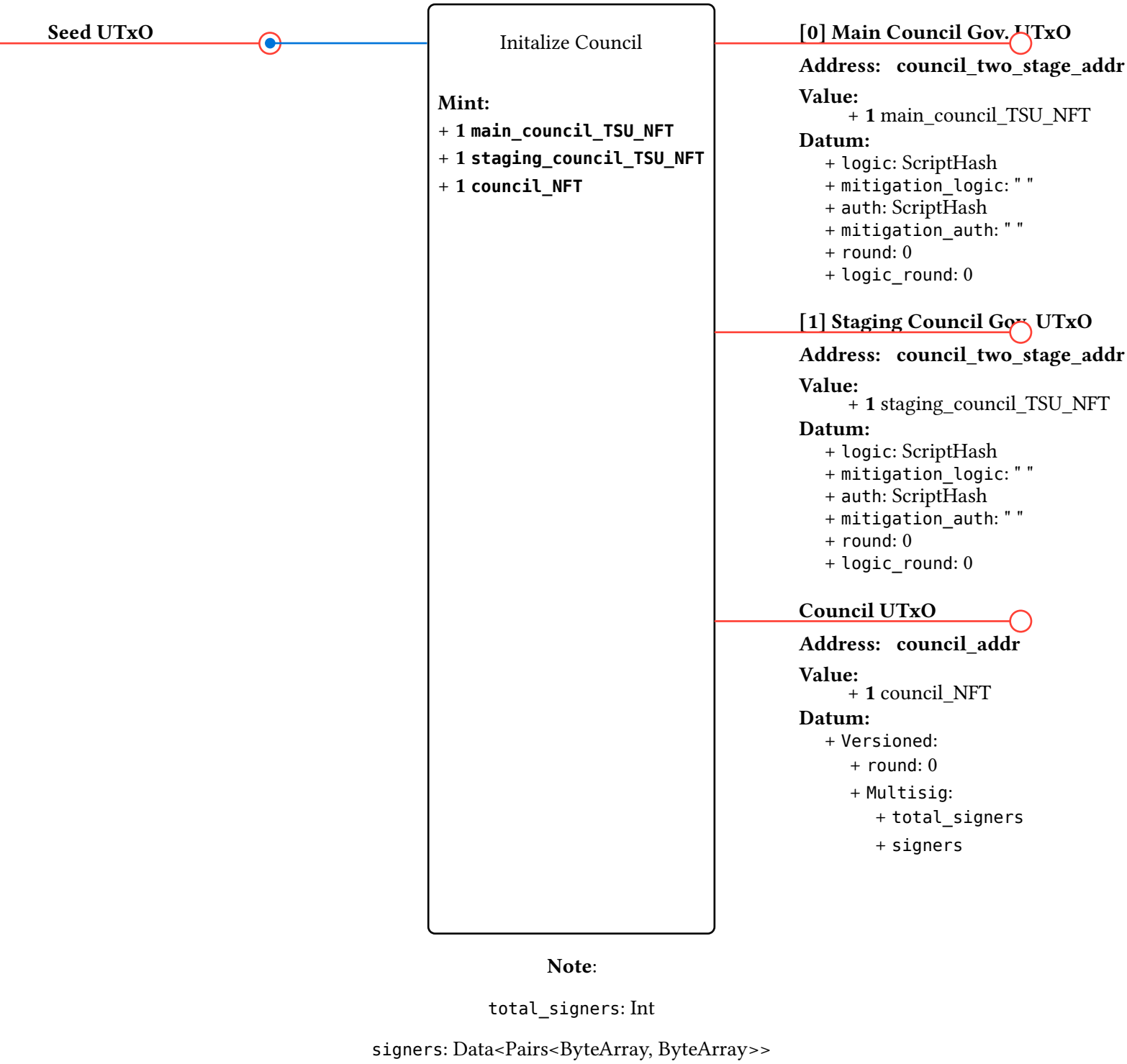


Figure 12: Initalize Council transaction

3.b.b.b - Invoke Council with Initial Council Logic

The purpose of this transaction is to change the composition of the Council. Approval by the Council and the Technical Authority is required. Thresholds are determined by the Council Update Threshold

UTxO (`thresholds.ak:main_council_update_threshold()`) that must be present as a reference input.

Analog transactions exists for the Technical Authority and the Federated Authority. Thresholds are determined, respectively, by the Tech Auth Update Threshold UTxO (`thresholds.ak:main_tech_auth_update_threshold()`) and the Fed. Auth Update Threshold UTxO (`thresholds.ak:main_federated_ops_update_threshold()`)

Involved scripts:

- Spend Main Council UTxO (`council_forever` script).
- "Logic" withdrawal: read from Main Governance UTxO datum. It is `permissioned.ak:council_logic()` script.
- "Mitigation logic" withdrawal: if not empty, read from Main Governance UTxO datum.
- Mint "N_OF_K_TECH_AUTH": required by council logic. Built from:
  - `signers`: from Tech Auth UTxO ref. input.
  - `tech_auth_numerator`: from Main Council Gov. Threshold UTxO.
  - `tech_auth_denominator`: from Main Council Gov. Threshold UTxO.
- Mint "N_OF_K_COUNCIL": required by council logic. Built from:
  - `signers`: from Council UTxO ref. input.
  - `council_numerator`: from Main Council Gov. Threshold UTxO.
  - `council_denominator`: from Main Council Gov. Threshold UTxO.

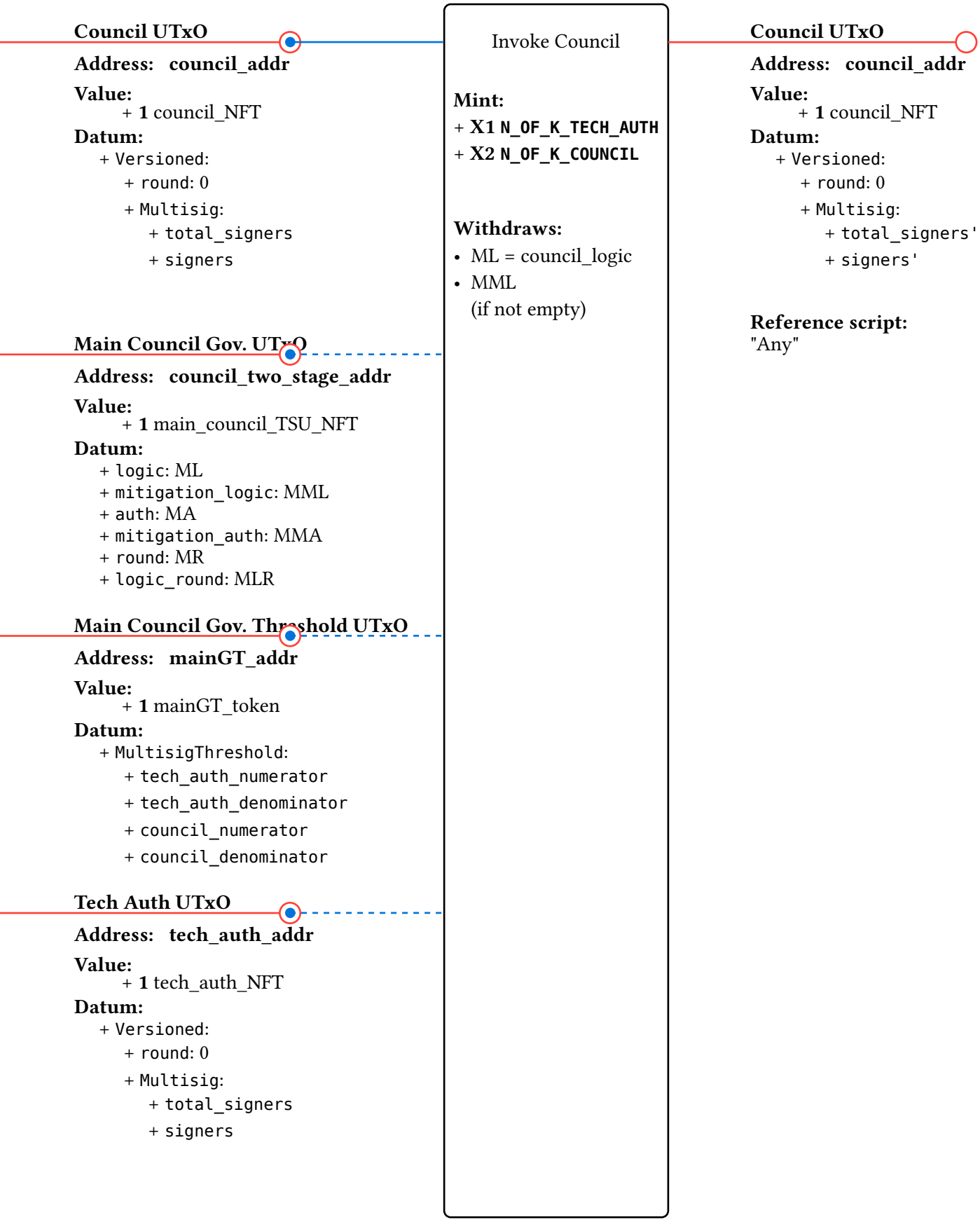


Figure 13: Invoke Council transaction with logic given by given `permissioned.ak:council_logic()`.

3.b.b.c - Upgrade Council

All transactions are similar to the upgrading transactions for the Reserve contract:

- Staging:
  - Stage new logic
  - Stage new auth logic
  - Stage new mitigation logic
  - Stage new auth mitigation logic
- Promoting:
  - Promote staged logic
  - Promote staged auth logic
  - Promote staged mitigation logic
  - Promote staged auth mitigation logic

3.b.c - Other upgradable contracts

3.b.c.a - Techincal Authority

All transactions are similar to the Permissioned Council.

3.b.c.b - Federated Ops

All transactions are similar to the Permissioned Council.

3.b.c.c - Terms and Conditions

All transactions are similar to the Permissioned Council.

3.b.c.d - Illiquid Circulation Supply

All transactions are similar to the Reserve.

3.b.d - Main/Staging Threshold

Involved scripts:

- `main_gov_threshold` (`mainGT`) script.
- `staging_gov_threshold` (`stagingGT`) script.

Involved Addresses:

- mainGT_addr = { payment: `mainGT`, staking: any }
- stagingGT_addr = { payment: `stagingGT`, staking: any }

Involved Assets:

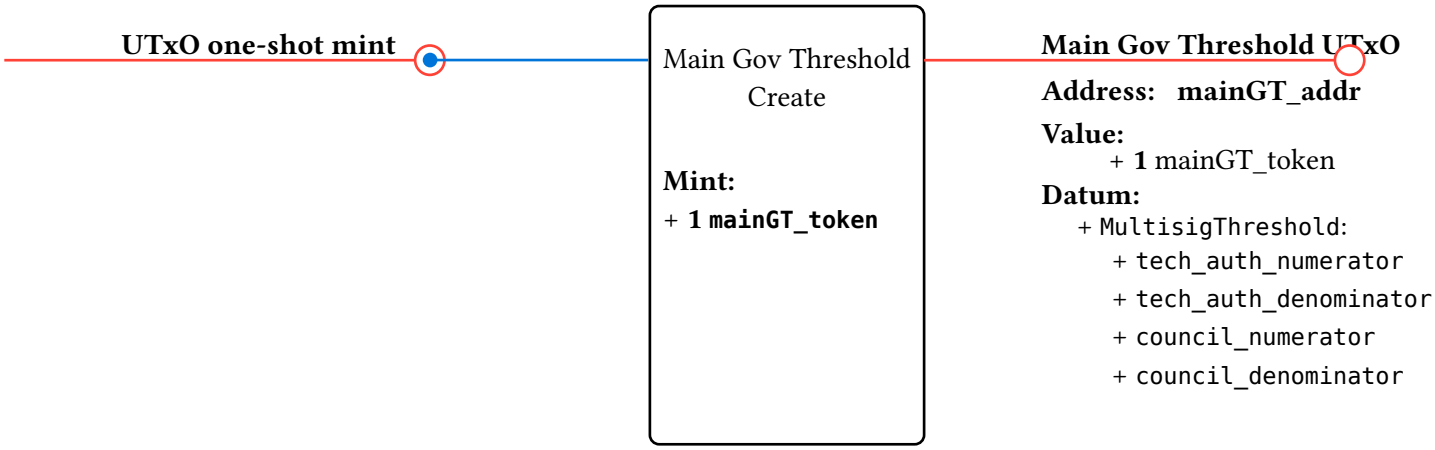
- mainGT_token = `mainGT.""`
- staingGT_token = `stagingGT.""`
- tech_auth_proof_token = `tech_auth_native_script_hash.`any
- council_proof_token = `council_native_script_hash.`any

The Native script proof of proper authorization, it's built during the validation using the `Multisig` information from the corresponding UTxO together with the threshold information. We have two native scripts: `tech_auth_native_script` and `council_native_script`.

3.b.d.a - Main Governance Threshold: Create

Involved scripts: `main_gov_threshold` (`mainGT`) script.

- Mint
  - Redeemer: `()`



Note:

The "UTxO one-shot mint" is parameter of the `mainGT` script

$$\texttt{tech_auth_numerator} < \texttt{tech_auth_denominator}$$

$$\texttt{council_numerator} < \texttt{council_denominator}$$

$$-1 < \texttt{technical_auth_numerator}$$

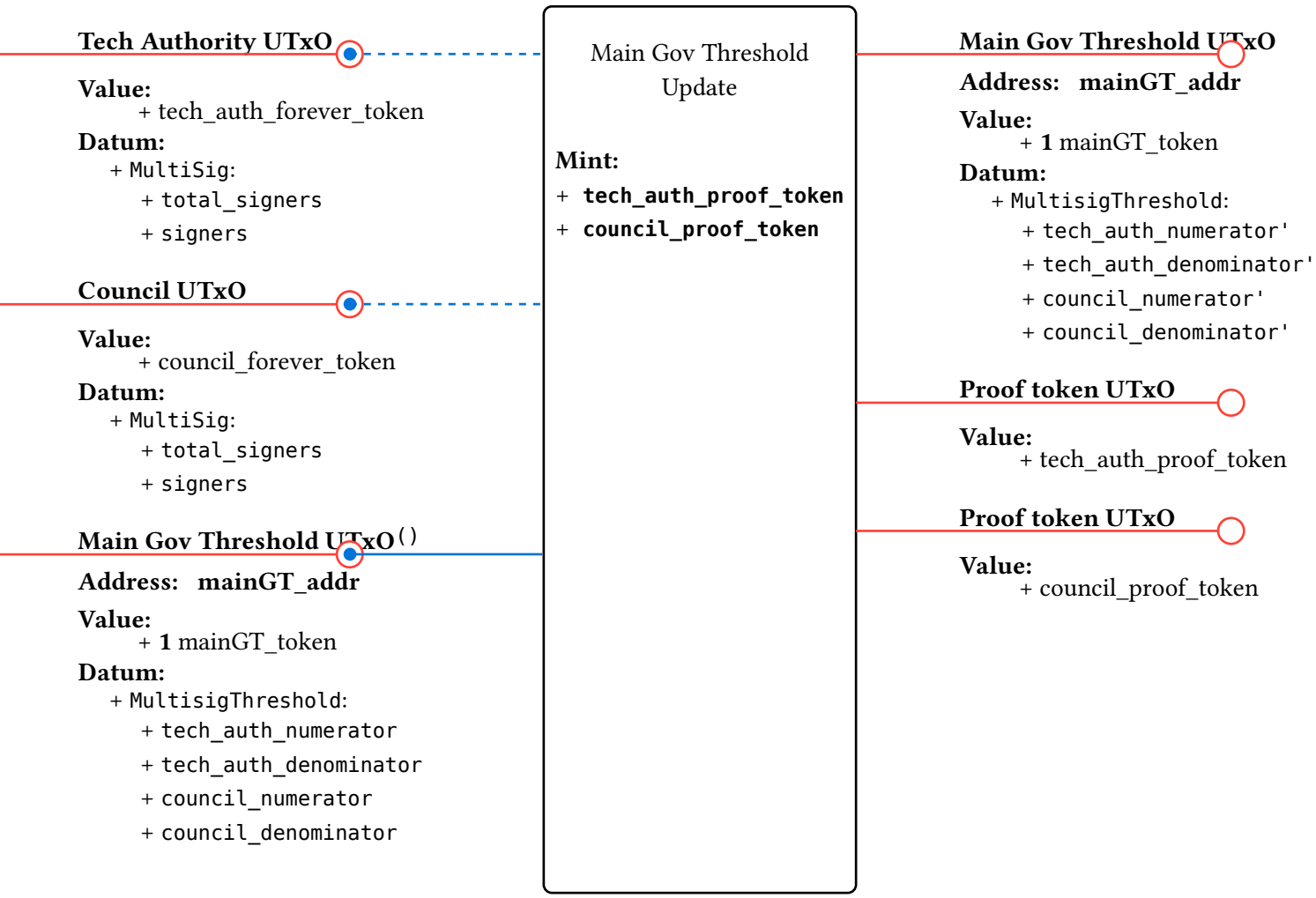
$$-1 < \texttt{council_numerator}$$

Figure 14: Main governance threshold UTxO creation

3.b.d.b - Main Governance Threshold: Update

Involved scripts: `main_gov_threshold` (`mainGT`) script.

- Spend
  - Redeemer: `()`



Note:

`tech_auth_numerator' < tech_auth_denominator'`

`council_numerator' < council_denominator'`

$-1 <$ `technical_auth_numerator'`

$-1 <$ `council_numerator'`

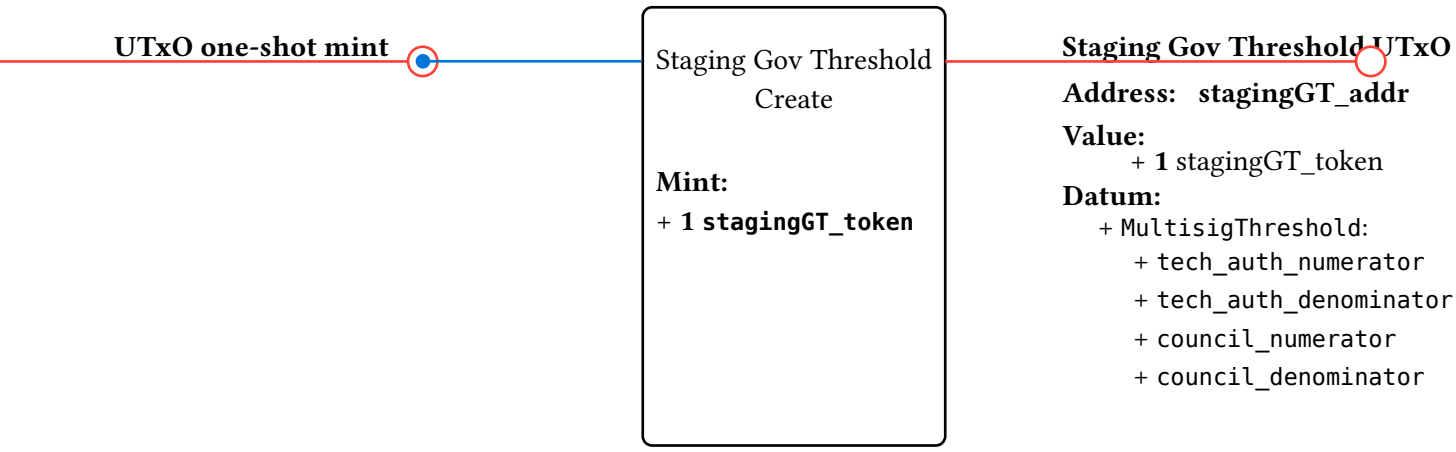
The `tech_auth_native_script_hash` is built using the `MultiSig` information placed in the "Tech Authority UTxO" together with the `tech_auth_numerator` and `tech_auth_denominator`. Analogous for the `council_native_script_hash`

Figure 15: Main governance threshold UTxO update

3.b.d.c - Staging Governance Threshold: Create

Involved scripts: `staging_gov_threshold` (`stagingGT`) script.

- Mint
  - Redeemer: `()`



Note:

The "UTxO one-shot mint" is parameter of the `stagingGT` script

`tech_auth_numerator < tech_auth_denominator`

`council_numerator < council_denominator`

0 < `tech_auth_numerator`

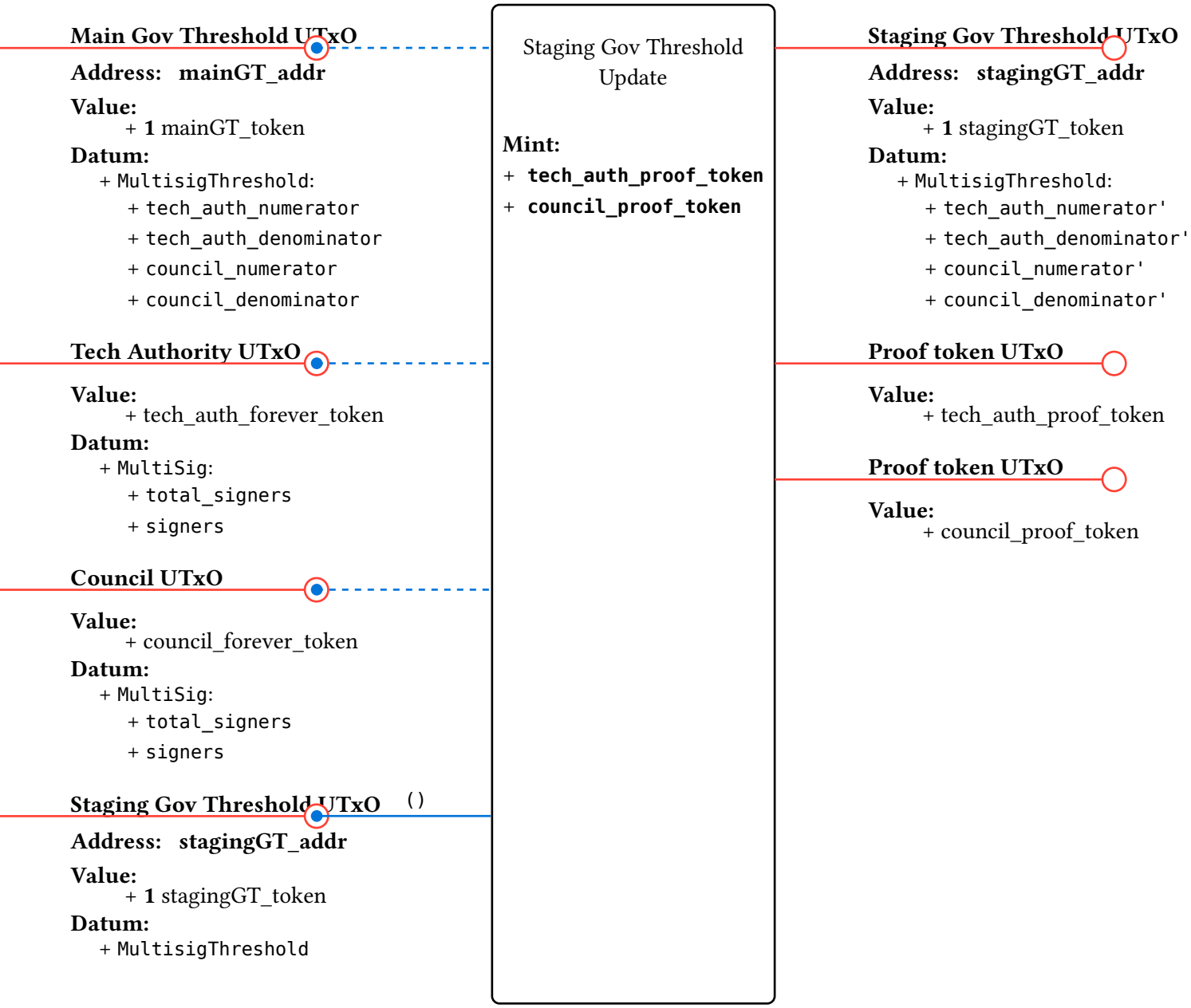
0 < `council_numerator`

Figure 16: Staging governance threshold UTxO creation

3.b.d.d - Staging Governance Threshold: Update

Involved scripts: `staging_gov_threshold` (`stagingGT`) script.

- Spend
  - Redeemer: `()`



Note:

The datum information of the "Staging Gov Threshold" UTxO isn't used

`tech_auth_numerator' < tech_auth_denominator'`

`council_numerator' < council_denominator'`

0 < `technical_auth_numerator'`

0 < `council_numerator'`

The `tech_auth_native_script_hash` is built using the `MultiSig` information placed in the "Tech Authority UTxO" together with the `tech_auth_numerator` and `tech_auth_denominator`. Analogous for the `council_native_script_hash`

Figure 17: Staging governance threshold UTxO update

3.b.e - Other threshold contracts

3.b.e.a - Concil Update Threshold

Similar to Staging Gov. Threshold.

3.b.e.b - Technical Authority Update Threshold

Similar to Staging Gov. Threshold.

3.b.e.c - Federated Authority Update Threshold

Similar to Staging Gov. Threshold.

3.b.f - cNight Generates Dust

Involved scripts:

- `cnight_generates_dust` (`cNGD`) script.

Involved Addresses:

- cNGD_addr = { payment: `cNGD`, staking: any }

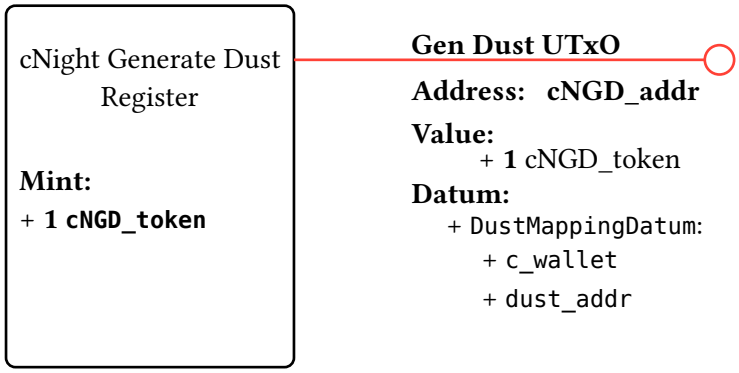
Involved Assets:

- cNGD_token = `cNGD.""`

3.b.f.a - Register

Involved scripts: `cnight_generates_dust` (`cNGD`) script.

- Mint
  - Redeemer: `Create`



Note:

If `c_wallet` is a `PubKey` credential, then that credential must sign the transaction

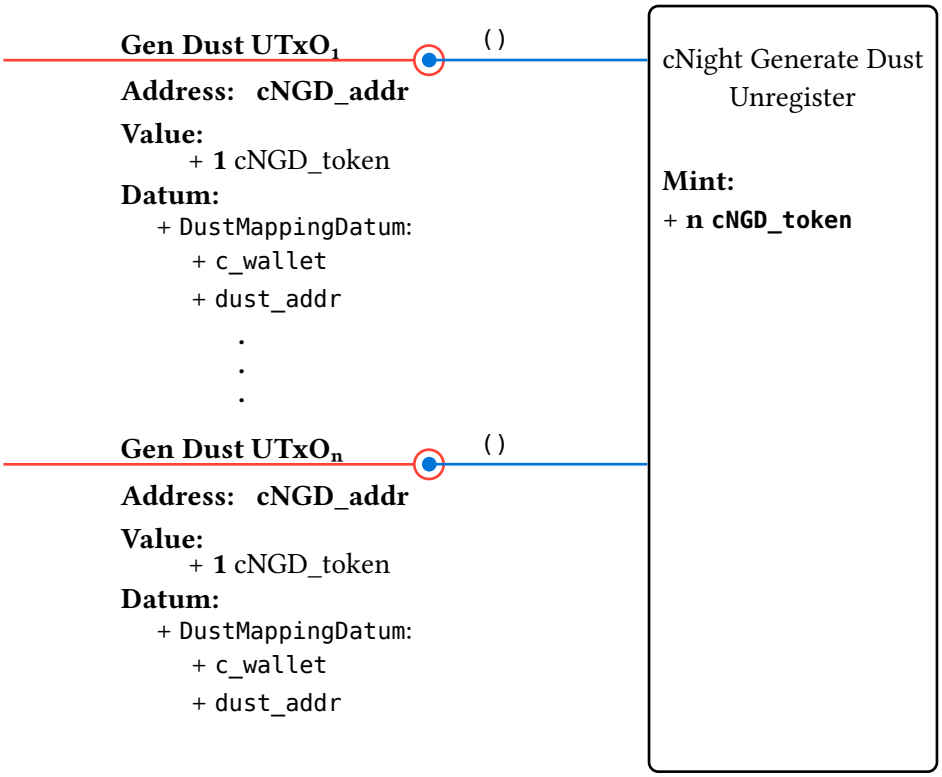
If `c_wallet` is a `Script` credential, then a withdrawal action with that credential must be included in the transaction

Figure 18: Address registration for Dust production

3.b.f.b - Unregister

Involved scripts: `cnight_generates_dust` (`cNGD`) script.

- Mint
  - Redeemer: `Burn`
- Spend
  - Redeemer: `()`



Note:

The `c_wallet` and `dust_addr` may differ across all the inputs

If `c_wallet` is a `PubKey` credential, then that credential must sign the transaction

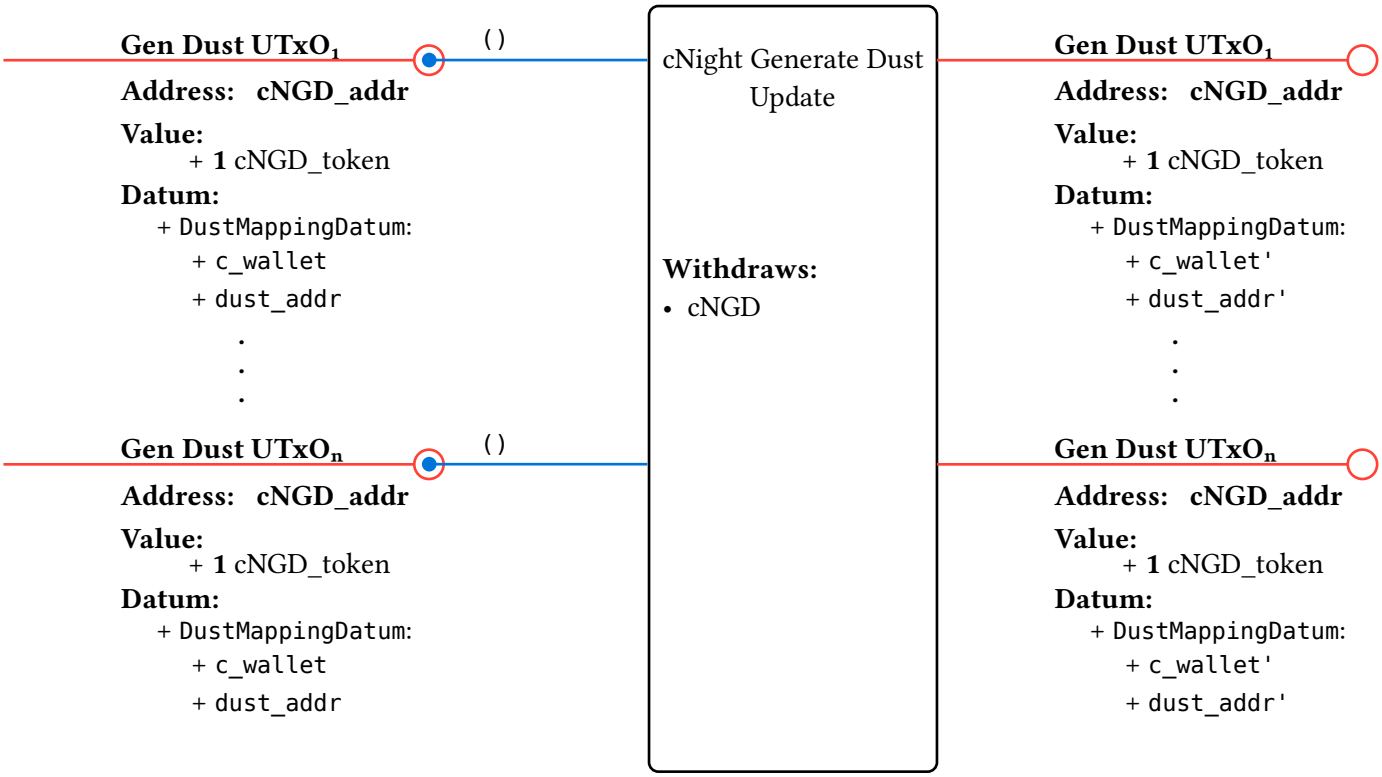
If `c_wallet` is a `Script` credential, then a withdrawal action with that credential must be included in the transaction

Figure 19: Address unregistration for Dust production

3.b.f.c - Update

Involved scripts: `cnight_generates_dust` (`cNGD`) script.

- Spend
  - Redeemer: `()`
- Withdraw
  - Redeemer: `()`



Note:

The minting or burning of `cNGD_tokens` is prohibited

The `c_wallet` and `dust_addr` may differ across all the inputs

The `c_wallet'` and `dust_addr'` may differ across all the outputs

If `c_wallet` is a `PubKey` credential, then that credential must sign the transaction

If `c_wallet` is a `Script` credential, then a withdrawal action with that credential must be included in the transaction

Figure 20: Dust production information update

4 - Findings

ID	Title	Severity	Status
MRC-001	Merge logic allows the creation of unspendable outputs	Critical	Resolved
MRC-002	“cNIGHT Generates Dust” NFTs can be extracted from contract	Critical	Resolved
MRC-003	Too restrictive logic round version check	Critical	Resolved
MRC-201	Missing datum checks in the Main Gov Threshold UTxO	Minor	Resolved
MRC-202	Merge logic allows the inclusion of arbitrary assets	Minor	Resolved
MRC-203	Permissioned contracts can contain incorrect datum	Minor	Resolved
MRC-204	Staging Auth Mitigation Script involves running previous undesired AMS	Minor	Resolved
MRC-301	No inline documentation	Info	Resolved
MRC-302	Modularizable code for value checks	Info	Resolved
MRC-303	Multisig authorization approach mints useless tokens	Info	Acknowledged

4.a - MRC-001 Merge logic allows the creation of unspendable outputs

Category	Commit	Severity	Status
Bug	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Critical	Resolved

4.a.a - Description

The `logic_merge()` function (`lib/logic/script.ak`) provides reusable validation logic for merging the value of all inputs locked by a given script into a single output, which is also paid to the same script. This function is intended to serve as the initial logic for both the Reserve and the Illiquid Circulation Supply.

The validation logic does not perform any checks on the output's datum. Therefore, it is possible to include an arbitrary datum hash, making the output unspendable and permanently locking the merged value.

4.a.b - Recommendation

Enforce that the resulting output includes an inline datum.

4.a.c - Resolution

Resolved in commit `67ed9c2dbf4cf6a26431fff3e8f9537bc5c1dcd7`.

4.b - MRC-002 “cNIGHT Generates Dust” NFTs can be extracted from contract

Category	Commit	Severity	Status
Bug	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Critical	Resolved

4.b.a - Description

“cNIGHT Generates Dust” contracts contain a validity NFT. When spending them for a no-mint update, a withdrawal is invoked to check that all spent instances have a corresponding continuation output.

However, no addresses are checked for the continuation outputs, so they can be paid to any address, including the enclosed NFTs.

4.b.b - Recommendation

When folding over the outputs, extract the `address` fields and check that the payment part has the correct value.

4.b.c - Resolution

Resolved in commit `ac3555e3ad52ee099a329c84284844ad43d04ac9`.

4.c - MRC-003 Too restrictive logic round version check

Category	Commit	Severity	Status
Bug	`c0b2dc93bb8766c7906618d22a18f706452f245f`	Critical	Resolved

4.c.a - Description

The `check_for_multisig` is used when executing the following validations:

- main and staging governance auth
- council initial logic
- tech auth initial logic
- every threshold spending-related validation

The function enforces the logic round version of the council, and tech auth to be `0`. The problem is their logic round version could (and most probably would) change. Thus, causing the protocol to stall.

4.c.b - Recommendation

The logic round version in the `check_for_multisig` function must be ignored.

4.c.c - Resolution

Resolved in commit `5435cd1ee94f32ee08ef7189e209ea03177c779d`.

4.d - MRC-201 Missing datum checks in the Main Gov Threshold UTxO

Category	Commit	Severity	Status
Bug	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Minor	Resolved

4.d.a - Description

The minting action in the `main_gov_threshold` validator ensures the proper initialization of the Main Governance Threshold UTxO. The validation over this UTxO involves checking the address, value and datum are correct.

In particular, the datum must have type `MultisigThreshold`, which has four fields: `technical_auth_numerator`, `technical_auth_denominator`, `council_numerator` and `council_denominator`. The current checks ensure that each denominator is greater than its numerator, but there is no verification to ensure that the numerators are positive. Each of these pairs (numerator and denominator) is used to calculate the minimum number of signatures required (by a Native `script_n_of_k`) to perform a governance action over the protocol.

If the numerators are negative, the minimum number of signatures can be achieved by providing zero of them. Thus, performing a governance action without proper authorization.

4.d.b - Recommendation

Add validations that ensure the numerators are always positives. The `threshold:validation` function could be used.

4.d.c - Resolution

Resolved in commit `3bd3d7b556f5f9aadc9d85a3a49d9697ee065332`.

4.e - MRC-202 Merge logic allows the inclusion of arbitrary assets

Category	Commit	Severity	Status
Robustness	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Minor	Resolved

4.e.a - Description

The `logic_merge()` function (`lib/logic/script.ak`) provides reusable validation logic for merging the value of all inputs locked by a given script into a single output, which is also paid to the same script. This function is intended to serve as the initial logic for both the Reserve and the Illiquid Circulation Supply.

Since only the address is checked for the merged inputs, it is possible to include any asset in the merged value. An attacker could exploit this by consuming the main Reserve/ICS UTxO, identified by the "forever" NFT, and creating a continuation output that merges this NFT with assets unrelated to the protocol.

This could clog the main UTxO, potentially preventing future merges of protocol-relevant assets. Additionally, the injected assets might have problematic token names, such as those containing offensive language.

4.e.b - Recommendation

Only allow merging relevant assets, such as the main NFT and CNIGHT tokens.

4.e.c - Resolution

Resolved in commit `67ed9c2dbf4cf6a26431fff3e8f9537bc5c1dcd7`.

4.f - MRC-203 Permissioned contracts can contain incorrect datum

Category	Commit	Severity	Status
Bug	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Minor	Resolved

4.f.a - Description

All permissioned contracts lock a datum containing a `signers` field, which map Cardano keys (encoded as native scripts) to byte strings.

The type of this mapping is:

```
Data<Pairs<NativeScriptSigner, Sr25519PubKey>>
```

where both `NativeScriptSigner` and `Sr25519PubKey` are synonyms for `ByteArray`.

The function `validate_multisig_structure()` (defined in `permissioned.ak`) is used to verify the correct structure of the `signers` field. Because the mapping is wrapped into a `Data` type, type casting does not guarantee that the internal structure of the mapping is correct. Instead, validation is performed by explicitly reconstructing the expected mapping, using the redeemer to supply the correct list of key–value pairs.

However, in the current implementation, the second component of each pair is always treated as `Data`. There is no validation to ensure that it has the expected type `Sr25519PubKey` (i.e., `ByteArray`). As a result, this field may contain arbitrary data, contradicting the type definition.

4.f.b - Recommendation

Add validation logic to ensure that the second component of each pair can be correctly decoded as an `Sr25519PubKey`.

4.f.c - Resolution

Resolved in commit `f11d27828666e887fb495a85242edf9b8a78192f`.

4.g - MRC-204 Staging Auth Mitigation Script involves running previous undesired AMS

Category	Commit	Severity	Status
Bug	`7b6838e4e63b5002c2ca2833421d47cdd766b9c1`	Minor	Resolved

4.g.a - Description

In upgradable contracts, the Auth Mitigation Script (AMS) can be promoted to the Main UTxO only once. Before promotion, the AMS must be staged and tested. If the staged AMS does not work as expected, it can be replaced by staging a new version. Indeed, staging a new AMS over a previous one clearly indicates that the old version is undesired.

However, in the current implementation validating the replacement of a staged AMS by staging a new one involves running the old AMS. More specifically, it is checked that both the previously staged auth script (AS) and AMS are running, or alternatively that the main auth script is running. It is possible to avoid the old AMS by using the main auth, but the main auth may be much more restrictive and difficult to satisfy, slowing down the process of implementing a correct AMS.

4.g.b - Recommendation

In our view, the old staged AMS should play no role when staging a new one. We recommend removing the requirement to run the old AMS in the validation of this action.

4.g.c - Resolution

Resolved in commit `ca94923280854b6ec765f1d74a27da4d3b75d1e4`.

4.h - MRC-301 No inline documentation

Category	Commit	Severity	Status
Documentation	`16e8f5512c74cdea97f44f519dd1aded790ac812`	Info	Resolved

4.h.a - Description

The source code doesn't have any inline documentation.

4.h.b - Recommendation

Add inline documentation for all functions and data types.

4.h.c - Resolution

Resolved in commit `acd99fce064b9781e081e3a00a78b156af9ed625`.

4.i - MRC-302 Modularizable code for value checks

Category	Commit	Severity	Status
Quality	`16e8f5512c74cdea97f44f519dd1aded790ac812`	Info	Resolved

4.i.a - Description

A frequent check used in the validators is to ensure that a value contains only a specific NFT and no other assets. Currently, this check is implemented in a low-level manner for optimization purposes. The same code is repeated every time the check is needed. This repetition increases code size and reduces readability, making validators longer and harder to maintain.

4.i.b - Recommendation

Modularize this check into a function. For instance, in `two-stage-upgrade.ak` there are at least seven places where the function could be used. Additionally, the functions `expected_main_nft_id` and `expected_staging_nft_id` could be removed.

4.i.c - Resolution

Resolved in commit `9569896ea7eb9c263c5d94499d5d967625c413f7`.

4.j - MRC-303 Multisig authorization approach mints useless tokens

Category	Commit	Severity	Status
Design	16e8f5512c74cdea97f44f519dd1aded790ac812	Info	Acknowledged

4.j.a - Description

The multisig authorization approach relies on the execution of native scripts to verify the presence of a minimum number of required signatures. For practical reasons, these native scripts are executed as minting policies, as the corresponding ledger rules are simpler than those of other possible script execution mechanisms.

However, this approach has the side effect of minting tokens that have no use beyond the minting event. The presence of such purposeless tokens reduces the understandability of the protocol and can be considered an anti-pattern within the Cardano paradigm.

Moreover, these tokens will pollute the network UTxOs, as they must be paid somewhere when minted. Since the validation allows both minting and burning, a possible workaround to mitigate this pollution is to alternately mint and burn the tokens.

4.j.b - Recommendation

Replace the minting policies with alternative script execution mechanisms. Under the current ledger rules, a viable option is to execute the scripts as stake credential registrations, including the corresponding de-registrations within the same transaction.

Starting with the upcoming Dijkstra era, a new execution mechanism called “guarding” will be the exact fit for this use.

4.j.c - Resolution

The project team has acknowledged this finding. Its resolution has been postponed to a future version of the project.

5 - Minor issues

In this section we list some issues we found that do not qualify as findings such as typos, coding style, naming, etc. We used the Github issues system to report them.

- Issue #25: document trick to include cnight policy in the script hash
- Issue #27: forever_contract: replace three first parameters with ScriptContext
- Issue #28: validate_running: change auth_cred parameter to ScriptHash type
- Issue #33: typo in filename iliquid_circulation_supply.ak: should be "illiquid"
- Issue #34: unnecessary special case for MitigationAuth
- Issue #35: indicate return types for validate_main_update and validate_staging_update
- Issue #38: redundant call to un_b_data: it is already a ByteString
- Issue #39: threshold_validation: datum validation can be done internally
- Issue #40: repeated code for checking c_wallet authentication
- Issue #41: Add NIGHT token name to the config
- Issue #42: Minor pattern matching simplification
- Issue #43: Strictest value pattern matching
- Issue #47: THD-4 and THD-5 inconsistent with code

A Appendix

A.1 Terms and Conditions of the Commercial Agreement

A.1.1 Confidentiality

Both parties agree, within a framework of trust, to discretion and confidentiality in handling the business. This report cannot be shared, referred to, altered, or relied upon by any third party without Txpipe LLC, 651 N Broad St, Suite 201, Middletown registered at the county of New Castle, written consent.

The violation of the aforementioned, as stated supra, shall empower TxPipe to pursue all of its rights and claims in accordance with the provisions outlined in Title 6, Subtitle 2, Chapter 20 of the Delaware Code titled "Trade Secrets,", and to also invoke any other applicable law that protects or upholds these rights.

Therefore, in the event of any harm inflicted upon the company's reputation or resulting from the misappropriation of trade secrets, the company hereby reserves the right to initiate legal action against the contractor for the actual losses incurred due to misappropriation, as well as for any unjust enrichment resulting from misappropriation that has not been accounted for in the calculation of actual losses.

A.1.2 Service Extension and Details

This report does not endorse or disapprove any specific project, team, code, technology, asset or similar. It provides no warranty or guarantee about the quality or nature of the technology/code analyzed.

This agreement does not authorize the client Shielded Technologies to make use of the logo, name, or any other unauthorized reference to Txpipe LLC, except upon express authorization from the company.

TxPipe LLC shall not be liable for any use or damages suffered by the client or third-party agents, nor for any damages caused by them to third parties. The sole purpose of this commercial agreement is the delivery of what has been agreed upon. The company shall be exempt from any matters not expressly

covered within the contract, with the client bearing sole responsibility for any uses or damages that may arise.

Any claims against the company under the aforementioned terms shall be dismissed, and the client may be held accountable for damages to reputation or costs resulting from non-compliance with the aforementioned provisions. **This report provides general information and is not intended to constitute financial, investment, tax, legal, regulatory, or any other form of advice.**

Any conflict or controversy arising under this commercial agreement or subsequent agreements shall be resolved in good faith between the parties. If such negotiations do not result in a conventional agreement, the parties agree to submit disputes to the courts of Delaware and to the laws of that jurisdiction under the powers conferred by the Delaware Code, TITLE 6, SUBTITLE I, ARTICLE 1, Part 3 § 1-301. and Title 6, SUBTITLE II, chapter 27 §2708.

A.1.3 Disclaimer

The audit constitutes a comprehensive examination and assessment as of the date of report submission. The company expressly disclaims any certification or endorsement regarding the subsequent performance, effectiveness, or efficiency of the contracted entity, post-report delivery, whether resulting from modification, alteration, malfeasance, or negligence by any third party external to the company.

The company explicitly disclaims any responsibility for reviewing or certifying transactions occurring between the client and third parties, including the purchase or sale of products and services.

This report is strictly provided for ***informational purposes*** and reflects solely the due diligence conducted on the following files and their corresponding hashes using sha256 algorithm:

Filename: `./validators/permissioned.ak` **Hash**: `421d881c2b2213e9d36992ac7ad70b8f248ed45f4eb7996fbb6973933085b737`
Filename: `./validators/terms_and_conditions.ak` **Hash**: `8ceb56992fff658d3fea0a2e4961e166be311c2184225b2a6e0fa3c982f1e1da`
Filename: `./validators/thresholds.ak` **Hash**: `56a25b5a33cfa38334721633fff1aa3d612e6ea0f95717fc7f3235976d4e9898`
Filename: `./validators/reserve.ak` **Hash**: `41023f7e8c930a1062e944213c7f20c7fcaf41de10b3feb4f15e85e4ff49ff1d`
Filename: `./validators/cnight_generates_dust.ak` **Hash**: `73bd30bdc6b0815244bfd275fc410bdfdf54bbf83791439ebce6b2646e7c960a`
Filename: `./validators/illiquid_circulation_supply.ak` **Hash**: `aea215223f5a83d80dbed2e4b75a38aeb9debe08b67a22c0d5cd7b03e147e74d`
Filename: `./validators/gov_auth.ak` **Hash**: `86c0d191ef79e76b214b20318a4a03195d944f123894a41d0af4b7fe72349575`
Filename: `./lib/auth/types.ak` **Hash**: `01ba4719c80b6fe911b091a7c05124b64eeece964e09c058ef8f9805daca546b`
Filename: `./lib/auth/staging.ak` **Hash**: `3f9e3e358158ba5971e669ca48775261c12708b5758d33b11b65dbf4b390cb5e`
Filename: `./lib/auth/script.ak` **Hash**: `41bb28f6bfe176f55ed03d17110e73003029e4ab2fab3b29c96bbbd146996a00`
Filename: `./lib/upgradable/types.ak` **Hash**: `cb53a1b3e7d06d6cec9931c465831c9b94ff52fdc4d3520950b2b75cefc74305`

Filename: `./lib/upgradable/two-stage-upgrade.ak` **Hash**: `631f859073513b065acc82d2bada72a8aae1d3a1dd8887c88fc6a81b273f57ca`
Filename: `./lib/utils.ak` **Hash**: `ef137896d369de48874e320306defd49fe7297a5bfcf69400788b887360ff9fe`
Filename: `./lib/logic/script.ak` **Hash**: `02fb23277e68cd85b1e4c5db55c2a4a27756b830e375bc9e5974727e1d1f6ae6`
Filename: `./lib/forever/types.ak` **Hash**: `0c551a69ba2e7eaf9c7bfaf0a9f2a2d8c222376fb6c4f09e66e127d21d03a9a1`
Filename: `./lib/forever/script.ak` **Hash**: `3b6d57518e8d8c60c51ca7c07d7a6b0a0dbdc5c6b6882e204593b7abae77d40a`
Filename: `./lib/multisig/types.ak` **Hash**: `ef975b3e0028ffbec6cfd9201bf2e338b3f428d2087742b407252d513570a597`
Filename: `./lib/multisig/script.ak` **Hash**: `93eb23dc7216c0580cc4ca81096dba217881508e0095715ba4163b69784ec1d9`

TxPipe advocates for the implementation of multiple independent audits, a publicly accessible bug bounty program, and continuous security auditing and monitoring. Despite the diligent manual review processes, the potential for errors exists. TxPipe strongly advises seeking multiple independent opinions on critical matters. It is the firm belief of TxPipe that every entity and individual is responsible for conducting their own due diligence and maintaining ongoing security measures.

A.2 Issue Guide

A.2.1 Severity

Severity	Description
Critical	Critical issues highlight exploits, bugs, loss of funds, or other vulnerabilities that prevent the dApp from working as intended. These issues have no workaround.
Major	Major issues highlight exploits, bugs, or other vulnerabilities that cause unexpected transaction failures or may be used to trick general users of the dApp. dApps with Major issues may still be functional.
Minor	Minor issues highlight edge cases where a user can purposefully use the dApp in a non-incentivized way and often lead to a disadvantage for the user.
Info	Info are not issues. These are just pieces of information that are beneficial to the dApp creator. These are not necessarily acted on or have a resolution, they are logged for the completeness of the audit.

A.2.2 Status

Status	Description
Resolved	Issues that have been **fixed** by the **project** team.
Acknowledged	Issues that have been **acknowledged** or **partially fixed** by the **project** team. Projects can decide to not **fix** issues for whatever reason.
Identified	Issues that have been **identified** by the **audit** team. These are waiting for a response from the **project** team.

A.3 Revisions

This report was created using a git based workflow. All changes are tracked in a github repo and the report is produced using typst. The report source is available here. All versions with downloadable PDFs can be found on the releases page.

A.4 About Us

TxPipe is a blockchain technology company responsible for many projects that are now a critical part of the Cardano ecosystem. Our team built Oura, Scrolls, Pallas, Demeter, and we're the original home of Aiken. We're passionate about making tools that make it easier to build on Cardano. We believe that blockchain adoption can be accelerated by improving developer experience. We develop blockchain tools, leveraging the open-source community and its methodologies.

A.4.1 Links

- Website
- Email
- Twitter